

torch.autograd.backward#

<https://pytorch.org/docs/stable/generated/torch.autograd.backward.html#torch>

Description:

Compute the sum of gradients of given tensors with respect to graph leaves. The graph is differentiated using the chain rule. If any of tensors are non-scalar (i.e. their data has more than one element) and require gradient, then the Jacobian-vector product would be computed, in this case the function additionally requires specifying `grad_tensors`. It should be a sequence of matching length, that contains the “vector” in the Jacobian-vector product, usually the gradient of the differentiated function w.r.t. corresponding tensors (None is an acceptable value for all tensors that don’t need gradient tensors). This function accumulates gradients in the leaves - you might need to zero `.grad` attributes or set them to None before calling it. See Default gradient layouts for details on the memory layout of accumulated gradients. Using this method with `create_graph=True` will create a reference cycle between the parameter and its gradient which can cause a memory leak. We recommend using `autograd.grad` when creating the graph to avoid this. If you have to use this function, make sure to reset the `.grad` fields of your parameters to None after use to break the cycle and avoid the leak.

Function Signature

```
torch.autograd.backward(tensors, grad_tensors=None,  
retain_graph=None, create_graph=False, grad_variables=None,  
inputs=None)[source]#
```

Parameters

Notes & Warnings

NOTE

Note Using this method with `create_graph=True` will create a reference cycle between the parameter and its gradient which can cause a memory leak. We recommend using `autograd.grad` when creating the graph to avoid this. If you have to use this function, make sure to reset the `.grad` fields of your parameters to `None` after use to break the cycle and avoid the leak.

NOTE

Note If you run any forward ops, create `grad_tensors`, and/or call backward in a user-specified CUDA stream context, see Stream semantics of backward passes.

NOTE

Note When inputs are provided and a given input is not a leaf, the current implementation will call its `grad_fn` (even though it is not strictly needed to get this gradients). It is an implementation detail on which the user should not rely. See [pytorch/pytorch#60521](#) for more details.

Detailed Documentation

Documentation

Compute the sum of gradients of given tensors with respect to graph leaves.

The graph is differentiated using the chain rule. If any of tensors are non-scalar (i.e. their data has more than one element) and require gradient, then the Jacobian-vector product would be computed, in this case the function additionally requires specifying `grad_tensors`. It should be a sequence of matching length, that contains the “vector” in the Jacobian-vector product, usually the gradient of the differentiated function w.r.t. corresponding tensors (None is an acceptable value for all tensors that don’t need gradient tensors).

This function accumulates gradients in the leaves - you might need to zero `.grad` attributes or set them to None before calling it. See Default gradient layouts for details on the memory layout of accumulated gradients.

Using this method with `create_graph=True` will create a reference cycle between the parameter and its gradient which can cause a memory leak. We recommend using `autograd.grad` when creating the graph to avoid this. If you have to use this function, make sure to reset the `.grad` fields of your parameters to None after use to break the cycle and avoid the leak.

If you run any forward ops, create `grad_tensors`, and/or call backward in a user-specified CUDA stream context, see Stream semantics of backward passes.

When inputs are provided and a given input is not a leaf, the current implementation will call its `grad_fn` (even though it is not strictly needed to get this gradients). It is an implementation detail on which the user should not rely. See [pytorch/pytorch#60521](https://pytorch.org/docs/60521) for more details.

tensors (Sequence[Tensor] or Tensor or Sequence[GradientEdge] or GradientEdge) – Tensors of which the derivative will be computed.

grad_tensors (Sequence[Tensor or None] or Tensor, optional) – The “vector” in the Jacobian-vector product, usually gradients w.r.t. each element of corresponding

tensors. None values can be specified for scalar Tensors or ones that don't require grad. If a None value would be acceptable for all grad_tensors, then this argument is optional.

`retain_graph` (bool, optional) – If False, the graph used to compute the grad will be freed. Note that in nearly all cases setting this option to True is not needed and often can be worked around in a much more efficient way. Defaults to the value of `create_graph`.

`create_graph` (bool, optional) – If True, graph of the derivative will be constructed, allowing to compute higher order derivative products. Defaults to False.

`inputs` (Sequence[`Tensor`] or `Tensor` or Sequence[`GradientEdge`], optional) – Inputs w.r.t. which the gradient be will accumulated into `.grad`. All other Tensors will be ignored. If not provided, the gradient is accumulated into all the leaf Tensors that were used to compute the tensors.